Mock DS (DS blanc): OOP-Java

Practice exam written for revision, in the style of Dr. Soumaya Trabelsi's labs (PRISTINI 2026-2027). **Not an official exam.**
Covers: Ch1 basics · control structures & Scanner · classes/objects · constructors · `this` · access & encapsulation · scope · overloading · `static` / `final` · Labs 1, 2, 3.

Part	Content	Points	Suggested time
I	QCM: 20 questions (one correct answer each)	5 pts	20 min
II	Questions de réflexion (short written answers)	5 pts	25 min
III	Exercises (lab style): trace, find the errors, write the code	10 pts	75 min

Rules: on paper, no IDE, no notes, 2 hours. Then correct yourself with the **Corrigé** at the end. Every Java program in this file (exercises and solutions) was compiled and run with `javac` / `java`, and the outputs shown are the real outputs.

Part I: QCM (20 × 0.25 pt)

Circle the single correct answer.

1. The Java compiler `javac` transforms `Hello.java` into:
a) machine code for the processor b) bytecode in `Hello.class` c) an `.exe` file d) source code for the JVM

2. Which element executes the bytecode and makes Java portable?
a) javadoc b) the JDK compiler c) the JVM d) the garbage collector

3. A file contains `public class BankAccount { … }`. Its name must be:
a) `bankaccount.java` b) `BankAccount.class` c) `BankAccount.java` d) any name ending in `.java`

4. Which identifier follows the course conventions for a **constant**?
a) `maxSpeed` b) `MaxSpeed` c) `MAX_SPEED` d) `max_speed`

5. Which conversion is done **implicitly** (without a cast)?
a) `double → int` b) `int → double` c) `long → int` d) `float → long`

6. `int a = 9, b = 2; double r = a / b;` The value of `r` is:
a) 4.5 b) 4.0 c) 5.0 d) compilation error

7. Which declaration does **not** compile?
a) `double d = 1.5;` b) `float f = 1.5f;` c) `float f = 1.5;` d) `int i = (int) 1.5;`

8. `System.out.println("Total: " + 4 + 6);` displays:
a) `Total: 10` b) `Total: 46` c) `Total: 4 6` d) error

9. Which loop always executes its body **at least once**?
a) `for` b) `while` c) `do … while` d) none

10. In a `switch`, if a `case` has no `break`:
a) compilation error b) the switch stops c) execution continues into the following cases d) the `default` runs

11. To use the `Scanner` class you must import the package:

a) `java.lang` b) `java.io` c) `java.util` d) `java.scanner`

12. Which instruction reads a **character** with a Scanner `sc`?

a) `sc.nextChar()` b) `sc.next().charAt(0)` c) `sc.nextLine(0)` d) `sc.read()`

13. Which statement about a **constructor** is TRUE?

a) its return type is `void` b) it has the same name as the class and no return type c) it can be called several times on the same object d) a class can only have one

14. The class `Person` only defines `public Person(String n)`. The instruction `Person p = new Person();`:

a) creates a Person with name `null` b) creates a Person with name `""` c) causes a compilation error d) causes an error only at run time

15. With the default constructor, an attribute `boolean active;` and an attribute `String name;` are initialised to:

a) `true` and `""` b) `false` and `null` c) `0` and `null` d) they are not initialised

16. The instruction `this(0, 0);` inside a constructor:

a) can be placed anywhere in the constructor b) must be the first statement c) creates a new object d) is only allowed in `main`

17. An attribute declared `private` is accessible:

a) from every class b) from classes of the same package c) only from the methods of its own class d) from subclasses only

18. An attribute declared **without any modifier** (friendly) is accessible from:

a) its class only b) the classes of the same package c) every class d) subclasses in other packages

19. Which pair is **NOT** a valid overloading in the same class?

a) `void f(int x)` / `void f(double x)` b) `void f(int x)` / `void f(int x, int y)`

c) `int f(int x)` / `double f(int x)` d) `void f()` / `void f(String s)`

20. A `static` attribute:

a) can't be modified b) has one copy shared by all objects of the class c) is private by default d) is destroyed by the GC after each method

Part II: Questions de réflexion (5 pts)

Answer in 3–6 lines. Use a small code example when it helps. Answer 5 of the 7 (1 pt each), or all 7 for practice.

Q1. Explain the difference between a **class** and an **object**. Give a real-life example and the Java line that creates the object.

Q2. What is a **constructor**? Give 3 differences between a constructor and an ordinary method. What happens to the default constructor when you write your own?

Q3. In Lab 3, the junior developer wrote `myAccount.balance = 500.0;` in `main`. Explain why this doesn't compile, and why it is **better** to go through a method `deposit(double amount)` than to make `balance` public. Which OOP principle is this?

Q4. Give the **three uses** of the keyword `this` seen in class and labs, with one line of code for each.

Q5. Compare an **attribute** (instance variable), a **local variable** and a **static attribute**: where is each declared, how long does it live, does it have a default value?

Q6. Consider:

```
Person p1 = new Person("Ali", 20);
Person p2 = p1;
p2.age = 30;
System.out.println(p1.age);
```

What is displayed? How many objects exist in memory? Explain. What happens to the object if we then write `p1 = null; p2 = null;`?

Q7. In the guessing game (Lab 1 Ex5), the course asks for a `do … while` loop rather than a `while` loop. Why is it the natural choice here? Then explain why `m = (a + b) / 2;` gives `5.0` instead of `5.5` for `a = 4, b = 7` (with `double m`) and give two fixes.

Part III: Exercises (10 pts)

Exercise 1: What does this program display? — 2 pts

Write the exact output, line by line. Justify the lines marked ★ in one sentence each.

```java
public class Mystery {
    static int count = 0;
    int id;
    String label;

    Mystery(String label) {
        this.label = label;
        count++;
        id = count;
    }
    Mystery() {
        this("none");
    }
    void show() {
        System.out.println(id + ":" + label);
    }
    static int twice(int x)       { return x * 2; }
    static double twice(double x) { return x * 3; }

    public static void main(String[] args) {
        int a = 7, b = 2;
        double m = a / b;
        System.out.println(m);                        // ★ line 1
        System.out.println((double) a / b);           // line 2
        System.out.println(a % b + " " + a + b);      // ★ line 3

        Mystery x = new Mystery("A");
        Mystery y = new Mystery();
        Mystery z = x;
        z.label = "B";
        x.show();                                     // ★ line 4
        y.show();                                     // line 5
        System.out.println(Mystery.count);            // line 6
        System.out.println(twice(4) + " " + twice(4.0)); // ★ line 7

        int i = 10;
        do {
            System.out.print(i + " ");
            i += 5;
        } while (i < 10);
        System.out.println();                         // ★ line 8

        switch (b) {
            case 1:  System.out.print("one ");
            case 2:  System.out.print("two ");
            case 3:  System.out.print("three ");
                     break;
            default: System.out.print("other");
        }
        System.out.println();                         // ★ line 9
    }
}
```

Exercise 2: Find and fix the errors (Lab 3 style)

2 pts

A junior developer wrote this code. It contains **6 compilation errors** and **1 logic error** (the code compiles but gives a wrong result).
1) For each error: give the line, explain why it is an error. 2) Rewrite the corrected code, respecting encapsulation (add what is needed).

```java
class Product {
    private String name;
    private double price;
    public final double TVA = 0.19;

    public void Product(String name, double price) {
        name = name;
        price = price;
    }
    public double priceWithTax() {
        double total = price + price * TVA;
    }
}
public class Shop {
    public static void main(String[] args) {
        Product p = new Product("Pen", 2.5);
        p.price = 3.0;
        int qty = 2.5;
        float discount = 0.1;
        p.TVA = 0.2;
        System.out.println(p.priceWithTax() * qty);
    }
}
```

Exercise 3: Smart ATM (Lab 1 style: Scanner, switch, loops) — 2 pts

Context: a bank wants a console prototype of an ATM. Write a class `SmartATM` with only a `main` method:

1. Declare a constant `MAX_WITHDRAW` = 500.0 using `final`, a `balance` initialised to 1000.0 and a counter of successful operations.
2. Using a `do … while` loop, display the menu `D: Deposit | W: Withdraw | B: Balance | Q: Quit` and read the user's choice as a **character** with `Scanner`.
3. Use a `switch` on the choice:
   - `'D'`: read an amount; if it is > 0, add it to the balance (count the operation), otherwise display `"Invalid amount!"`.
   - `'W'`: read an amount; refuse it with a message if it is above `MAX_WITHDRAW`, above the balance, or ≤ 0; otherwise withdraw it (count the operation).
   - `'B'`: display the balance. `'Q'`: display `"Goodbye!"`. `default`: `"Invalid choice!"`.
4. The loop stops when the user types `Q`. Then display the number of successful operations.
5. Finally, using a `for` loop, display the balance after each of the next 3 months if it earns 1% interest per month.

Example run (user input in brackets): [B] → `Balance: 1000.0`, [D] [250], [W] [600] → `Limit is 500.0`, [W] [300], [X] → `Invalid choice!`, [Q] → `Goodbye! Successful operations: 2`, then `Month 1: 959.50`, `Month 2: 969.10`, `Month 3: 978.79`.

Exercise 4: The Smartphone class (Labs 2–3 style: encapsulation, constructors, `this`, overloading) — 2 pts

Context: you're coding the battery manager of a smartphone. Create a class `Smartphone`:

1. **Private attributes:** `brand` (String), `battery` (int, between 0 and 100), `appsOpened` (int).
2. A **parameterised constructor** `Smartphone(String brand, int battery)` that uses `this`; the battery must go through the setter (validation); `appsOpened` starts at 0.
3. A **default constructor** that delegates to the parameterised one with `this(...)` (`"Generic"`, 50) and then prints `"Default constructor called."`.

4. Getters `getBrand()`, `getBattery()`, and a setter `setBattery(int battery)`: below 0 → 0, above 100 → 100.
5. `charge(int percent)`: if percent > 0, increases the battery (never above 100). It **returns the current object** to allow chaining.
6. `openApp()`: if the battery is ≥ 5, consumes 5% and increments `appsOpened`; otherwise prints `"Battery too low!"`. Returns the current object.
7. **Overload** `openApp(int n)`: opens `n` apps by calling `openApp()`. Returns the current object.
8. `display()` prints: `Brand: Nova | Battery: 2% | Apps opened: 2`
9. A test class `TestPhone` must produce **exactly**:

```
Default constructor called.
Brand: Generic | Battery: 50% | Apps opened: 0
Battery too low!
Brand: Nova | Battery: 2% | Apps opened: 2
Brand: Nova | Battery: 95% | Apps opened: 3
```

Bonus question: in `main`, why does `p1.battery = 80;` not compile, and what should you write instead?

Exercise 5: Online shop (Lab 2 slot-machine + Lab 3 Ex3 style: objects inside objects, arrays of objects)

2 pts

1. `Product`: private `name` (String) and `price` (double), a constructor initialising both, and the getters.
2. `ShoppingCart`:
   - private attributes: an **array of Products** `items` and an int `count` (number of products in the cart);
   - a constructor `ShoppingCart(int capacity)` that creates the array;
   - `boolean addProduct(Product p)`: adds `p` if there is room and returns `true`; otherwise prints `"Cart is full: <name> not added"` and returns `false`;
   - `double getTotal()` and the **overloaded** `double getTotal(double discountPercent)`;
   - `int luckyDiscount()`: returns a random integer between 0 and 20 (inclusive) using `Math.random()`;
   - `printReceipt()`: prints each product (`- Keyboard : 40.0`) then `Total: …`.
3. `Customer`: private `name` and an attribute `cart` of type `ShoppingCart`, a constructor, and `displayProfile()` that prints the name, calls the cart's `printReceipt()` and prints the total after a 10% discount.
4. `TestShop`: a cart of capacity 3; add Keyboard 40.0, Mouse 20.0, Screen 140.0, Cable 5.0; create the customer "Ines" and display the profile. Expected output:

```
Cart is full: Cable not added
Customer: Ines
- Keyboard : 40.0
- Mouse : 20.0
- Screen : 140.0
Total: 200.0
Total after 10% discount: 180.0
```

Corrigé (correction): don't read before trying!

Part I: QCM answers

Q	Ans	Why	Q	Ans	Why
1	b	javac → portable bytecode (.class)	11	c	Scanner is in `java.util`
2	c	the JVM runs bytecode on every OS	12	b	there's no `nextChar()`
3	c	file name = public class name, case-sensitive	13	b	no return type, not even void
4	c	constants in uppercase (+ `final`)	14	c	default constructor gone once you write one
5	b	widening: int → long → float → double	15	b	false / null (0 for numbers)
6	b	9/2 = 4 (int division), then stored as 4.0	16	b	this(...) = first statement
7	c	1.5 is a double → lossy into float	17	c	private = same class only
8	b	left to right: "Total: 4" + 6 → "Total: 46"	18	b	friendly = package
9	c	condition tested at the end	19	c	return type alone can't differ
10	c	fall-through	20	b	class attribute, one shared copy

Part II: model answers

Q1. A class is a model (mould) describing a family of objects with the same attributes and methods; an object is a concrete **instance** of that class, with its own identity and state (attribute values). Example: the class `Car` describes every car (registration, color, speed, increaseSpeed()); my white BMW X1123 is one object: `Car bmw = new Car("X1123", "white", 110);`. One class → as many objects as we want.

Q2. A constructor is a special method that initialises the attributes of an object when it is created. Differences: (1) same name as the class, (2) no return type, not even `void`, (3) called automatically, only once per object, by `new` (a method is called explicitly, as many times as we want). If we write no constructor, Java provides a default one (no parameters, attributes at 0/false/null). As soon as we write one constructor, the default one disappears: `new Person()` then gives a compilation error unless we also write `Person()`.

Q3. `balance` is `private`: it can only be accessed by the methods of `BankAccount`; `main` is in another class, so the compiler says "balance has private access". Going through `deposit(amount)` lets the class **check** the value (`if (amount > 0)`), so no one can put a negative or absurd balance: the object protects the integrity of its data. A public attribute could be set to anything. This is **encapsulation** (data hidden, access through the object's interface = public methods).

Q4. (1) Remove ambiguity when a parameter has the same name as an attribute: `this.name = name;` (2) Call another constructor of the same class, as the first statement: `this("Unknown Model", "Petrol", 2023);` (3) Return (or pass) the current object, e.g. for method chaining: `return this;` → `s1.incrementAndReturnId().display();`

Q5.

	Attribute	Local variable	Static attribute
Declared	in the class, outside methods	inside a method/block	in the class, with `static`
Copies	one per object	one per method call	one for the whole class
Lives	as long as the object	until the end of its block	as long as the program
Default value	0 / false / null	none (must be initialised)	0 / false / null
Access	`obj.attr` / in methods	only in its block	`ClassName.attr`

Q6. It displays **30**. There is only **one** object: `new` was called once. `p2 = p1` copies the **reference** (address), not the object, so p1 and p2 point to the same object, and changing it through p2 is visible through p1. After `p1 = null; p2 = null;` no reference points to the object any more: it becomes unreachable and the **garbage collector** can destroy it (we can ask with `System.gc()`).

Q7. The player must make at least one guess before we can compare it with the secret number; `do … while` tests the condition at the end, so the body (ask, compare, count) always runs at least once. With `while` we would have to read a first guess before the loop (duplicated code). `(a + b) / 2`: both operands are `int`, so it's an integer division: 11 / 2 = 5 (decimal part cut); only then is 5 converted to 5.0 for `m`. Fixes: `m = (a + b) / 2.0;` or `m = (double) (a + b) / 2;`

Exercise 1: output and justification

```
3.0
3.5
1 72
1:B
2:none
2
8 12.0
10
two three
```

- ★ L1: `7 / 2` is int division = 3, then stored as 3.0.
- L2: the cast happens **before** the division: 7.0 / 2 = 3.5.
- ★ L3: `a % b` = 1; then string concatenation left to right: "1 " + 7 → "1 7", + 2 → "1 72" (not 9!).
- ★ L4: `z = x` copies the reference: same object, so `z.label = "B"` changes x. x was created first → id 1.
- L5: `new Mystery()` calls `this("none")`; count becomes 2 → id 2.
- L6: `count` is static: one shared counter, 2 objects created (z is not a new object).
- ★ L7: overloading: `twice(4)` → int version (4×2 = 8); `twice(4.0)` → double version (4.0×3 = 12.0).
- ★ L8: do…while runs once even though `i < 10` is false at the start.
- ★ L9: `b = 2` → enters case 2; no break → falls into case 3, then `break`.

Exercise 2: errors

Line	Error	Fix

6	`void` makes `Product(...)` an ordinary method, **not a constructor**. So only the default constructor exists (this triggers the error at line 16).	remove `void`
7–8	**Logic error**: `name = name` assigns the parameter to itself; the attributes stay `null`/0.0.	`this.name = name;` `this.price = price;`
12	missing return statement: the method must return a double.	`return total;`
16	constructor `Product(String, double)` doesn't exist (see line 6).	fixed by line 6
17	`price` has private access in Product.	add a setter `setPrice` with validation
18	possible lossy conversion from double to int.	`int qty = 2;` (or `double`)
19	0.1 is a double → lossy conversion to float.	`0.1f`
20	cannot assign a value to final variable TVA.	remove the line (a constant can't change)

```java
class Product {
    private String name;
    private double price;
    public final double TVA = 0.19;

    public Product(String name, double price) {
        this.name = name;
        this.price = price;
    }
    public String getName()  { return name; }
    public double getPrice() { return price; }
    public void setPrice(double price) {
        if (price > 0) {
            this.price = price;
        }
    }
    public double priceWithTax() {
        double total = price + price * TVA;
        return total;
    }
}
public class Shop {
    public static void main(String[] args) {
        Product p = new Product("Pen", 2.5);
        p.setPrice(3.0);
        int qty = 2;
        float discount = 0.1f;
        System.out.println(p.priceWithTax() * qty);
    }
}
```

Real output: `7.140000000000001` (3.0 × 1.19 × 2 = 7.14; the extra digits are normal floating-point rounding with doubles).

Exercise 3: SmartATM

```java
import java.util.Scanner;

public class SmartATM {
    public static void main(String[] args) {
        final double MAX_WITHDRAW = 500.0;
        double balance = 1000.0;
        int operations = 0;
        char choice;
        Scanner sc = new Scanner(System.in);

        do {
            System.out.println("D: Deposit | W: Withdraw | B: Balance | Q: Quit");
            System.out.print("Your choice: ");
            choice = sc.next().charAt(0);

            switch (choice) {
                case 'D':
                    System.out.print("Amount: ");
                    double dep = sc.nextDouble();
                    if (dep > 0) {
                        balance += dep;
                        operations++;
                    } else {
                        System.out.println("Invalid amount!");
                    }
                    break;
                case 'W':
                    System.out.print("Amount: ");
                    double wd = sc.nextDouble();
                    if (wd > MAX_WITHDRAW) {
                        System.out.println("Limit is " + MAX_WITHDRAW);
                    } else if (wd > balance) {
                        System.out.println("Insufficient balance!");
                    } else if (wd <= 0) {
                        System.out.println("Invalid amount!");
                    } else {
                        balance -= wd;
                        operations++;
                    }
                    break;
                case 'B':
                    System.out.println("Balance: " + balance);
                    break;
                case 'Q':
                    System.out.println("Goodbye!");
                    break;
                default:
                    System.out.println("Invalid choice!");
            }
        } while (choice != 'Q');

        System.out.println("Successful operations: " + operations);

        for (int month = 1; month <= 3; month++) {
            balance = balance * 1.01;
            System.out.printf("Month %d: %.2f%n", month, balance); // (extra) println is accepted too
        }
        sc.close();
    }
}
```

Grading points

`final` constant ✓ · Scanner + import ✓ · `next().charAt(0)` ✓ · do…while with `!= 'Q'` ✓ · switch with `break` and `default` ✓ · validations with if/else if ✓ · counter ✓ · for loop ✓. With `println` instead of `printf` you'd see `959.5`, `969.095`, `978.7859500000001`: also accepted.

Exercise 4: Smartphone

```
class Smartphone {
    private String brand;
    private int battery;
    private int appsOpened;

    public Smartphone(String brand, int battery) {
        this.brand = brand;
        setBattery(battery);          // validation goes through the setter
        this.appsOpened = 0;
    }

    public Smartphone() {
        this("Generic", 50);          // MUST be the first statement
        System.out.println("Default constructor called.");
    }

    public String getBrand() { return brand; }
    public int getBattery()  { return battery; }

    public void setBattery(int battery) {
        if (battery < 0) {
            this.battery = 0;
        } else if (battery > 100) {
            this.battery = 100;
        } else {
            this.battery = battery;
        }
    }

    public Smartphone charge(int percent) {
        if (percent > 0) {
            setBattery(battery + percent);   // capped at 100 by the setter
        }
        return this;
    }

    public Smartphone openApp() {
        if (battery >= 5) {
            battery -= 5;
            appsOpened++;
        } else {
            System.out.println("Battery too low!");
        }
        return this;
    }

    public Smartphone openApp(int n) {      // overloading: same name, different parameters
        for (int i = 1; i <= n; i++) {
            openApp();
        }
        return this;
    }

    public void display() {
        System.out.println("Brand: " + brand + " | Battery: " + battery
                           + "% | Apps opened: " + appsOpened);
    }
}

public class TestPhone {
    public static void main(String[] args) {
        Smartphone p1 = new Smartphone();          // Default constructor called.
```

```
        Smartphone p2 = new Smartphone("Nova", 12);
        p1.display();
        p2.openApp(3);                          // 12 → 7 → 2 → "Battery too low!"
        p2.display();
        p2.charge(200).openApp().display();     // 2+200 → capped 100 → 95, 3 apps
    }
}
```

Bonus: `battery` is `private`, so `TestPhone` can't access it ("battery has private access in Smartphone"). Write `p1.setBattery(80);`: the setter also checks the value stays in [0, 100].

Exercise 5: Online shop

```
class Product {
    private String name;
    private double price;

    public Product(String name, double price) {
        this.name = name;
        this.price = price;
    }
    public String getName()  { return name; }
    public double getPrice() { return price; }
}

class ShoppingCart {
    private Product[] items;
    private int count;

    public ShoppingCart(int capacity) {
        items = new Product[capacity];   // array of references, all null
        count = 0;
    }

    public boolean addProduct(Product p) {
        if (count == items.length) {
            System.out.println("Cart is full: " + p.getName() + " not added");
            return false;
        }
        items[count] = p;
        count++;
        return true;
    }

    public double getTotal() {
        double total = 0;
        for (int i = 0; i < count; i++) {        // count, not items.length (nulls!)
            total += items[i].getPrice();
        }
        return total;
    }

    public double getTotal(double discountPercent) {   // overloading
        return getTotal() - getTotal() * discountPercent / 100;
    }

    public int luckyDiscount() {
        return (int) (Math.random() * 21);       // 0..20 inclusive
    }

    public void printReceipt() {
        for (int i = 0; i < count; i++) {
            System.out.println("- " + items[i].getName() + " : " + items[i].getPrice());
        }
        System.out.println("Total: " + getTotal());
    }
}

class Customer {
    private String name;
    private ShoppingCart cart;                   // an object inside an object

    public Customer(String name, ShoppingCart cart) {
        this.name = name;
        this.cart = cart;
```

```
    }

    public void displayProfile() {
        System.out.println("Customer: " + name);
        cart.printReceipt();                    // calling the inner object's methods
        System.out.println("Total after 10% discount: " + cart.getTotal(10));
    }
}

public class TestShop {
    public static void main(String[] args) {
        ShoppingCart cart = new ShoppingCart(3);
        cart.addProduct(new Product("Keyboard", 40.0));
        cart.addProduct(new Product("Mouse", 20.0));
        cart.addProduct(new Product("Screen", 140.0));
        cart.addProduct(new Product("Cable", 5.0));   // 4th → cart full
        Customer c = new Customer("Ines", cart);
        c.displayProfile();
    }
}
```

⚠ **Classic mistakes in this exercise**

Looping to `items.length` instead of `count` → `NullPointerException` on empty slots. Forgetting `count++`. Writing `Math.random() * 20` (gives 0..19 only). Writing `(int) Math.random() * 21`: the cast applies to `Math.random()` first → always 0!

Self-assessment

If you lost points on…	Re-read in the cheat sheet
QCM 5–8, Ex1 lines 1–3	§2: conversions, integer division, string concatenation
QCM 9–12, Ex3	§3: loops, switch, Scanner
QCM 13–16, Q2, Q4, Ex4	§5 constructors, §6 `this`
QCM 17–18, Q3, Ex2	§7 access & encapsulation, §13 compile errors
QCM 19–20, Q5, Ex1 lines 6–7	§9 overloading, §10 static
Q6, Ex5	§4 references, §11 lab patterns (composition, arrays of objects)